

Queries Using Comparison Operators (\$eq, \$ne, \$gt, \$gte, \$lt, \$lte), Logical Operators (\$and, \$or, \$not, \$nor),

Selection Criteria and Queries with Projection.

Note : Use “ Orders “ dataset for below questions, dataset is in file “1_Orders.txt”

1. Find all orders where the status is "Shipped". `db.packages.find({status:{Seq:"Shipped"}})`
2. Find all orders **not** in the "Delivered" status. `db.packages.find({status:{$ne:"Delivered"}})`
3. Find orders with a price greater than \$500. `db.packages.find({price:{$gt:500}})`
4. Find orders with a price less than or equal to \$100. `db.packages.find({price:{$lte:100}})`
5. Find orders where the quantity is greater than or equal to 2. `db.packages.find({quantity:{$gte:2}})`
6. Find orders where the order_id is less than 1005. `db.packages.find({order_id:{$lt:1005}})`
7. Find orders where the category is "Electronics". `db.packages.find({category:{Seq:"Electronics"}})`
8. Find all orders where category is "Electronics" and status is "Shipped". `db.packages.find({$and:[{category:{Seq:"Furniture"}},{status:{Seq:"Delivered"}}]})`
9. Find orders where either category is "Furniture" or status is "Delivered". `db.packages.find({$or:[{category:{Seq:"Furniture"}},{status:{Seq:"Delivered"}}]})`
10. Find orders where category is **not** "Electronics". `db.packages.find({category:{$ne:"Electronics"}})`
11. Find orders that are **neither** in "Shipped" nor in "Delivered" status. `db.packages.find({$and:[{status:{$ne:"Shipped"}},{status:{$ne:"Delivered"}}]})`
12. Find orders where the price is **greater than 500** and status is **not "Cancelled"**.
`db.packages.find({$and:[{price:{$gt:500}},{status:{$ne:"Cancelled"}}]})`
13. Find orders where quantity is **either 1 or 2**. `db.packages.find({quantity:{$in:[1,2]}})`
14. Find orders where quantity is **not 1 or 2**.
`db.packages.find({quantity:{$in:[1,2]}})`
15. Find all orders placed after "2024-02-05T00:00:00Z". `db.packages.find({order_date: { $gt: "2024-02-05T00:00:00Z"}})`
16. Find orders where customer_name is either "Alice Smith" or "John Doe". `db.packages.find({$or:[{customer_name:"Alice Smith"},{customer_name:"John Doe"}]})`
17. Find orders where status is **either "Processing" or "Cancelled"**, and price is **less than \$500**.
`db.packages.find({$or:[{status:"Processing"},{status:"Cancelled"}],price:{$lt:500}})`
18. Find orders where status is "Delivered" and show only customer_name and product fields.
`db.packages.find({status:{Seq:"Delivered"}},{customer_name:1,product:1,_id:0})`

19. Find orders where price is greater than **\$300** and display only **order_id, price, and status**.

```
db.packages.find({price:{$gt:300}},{order_id:1,price:1,status:1,_id:0})
```

20. Find orders where category is **"Electronics"** and display all fields except **email**.

```
db.packages.find({category:{$eq:"Electronics"}},{email:0})
```

21. Find orders where quantity is **more than 1** and display only **customer_name, product, and quantity**.

22. Find orders where status is **"Shipped"** and price is greater than **\$50**, displaying only **order_id, product,**

and price.

```
db.packages.find({status:{$eq:"Shipped"},price:{$gt:50}},{order_id:1,product:1,price:1,_id:0})
```

23. Find all orders where the **status** field does not exist. `db.packages.find({status:{$exists:false}})`

24. Find orders where the **price** field is of type **double** (floating point number).

```
db.packages.find({price:{$exists:true},price:{$type:"double"}})
```

25. Find orders where the **order_id** field is of type **int**. `db.packages.find({order_id:{$type:"int"}})`

26. Find orders where the **order_date** field **exists** and is stored as a **string**.